

Introduction to High-Level Synthesis — Part I: Theory

- Introduction to High-Level Synthesis
 - หลักสูตรอบรมระดับเริ่มต้นถึงระดับกลาง (Beginner → Intermediate)
 - สารบัญ
- Part I — ภาคทฤษฎี
 - บทที่ 1 — High-Level Synthesis คืออะไร?
 - บทที่ 2 — กระบวนการทำงานของ HLS
 - บทที่ 3 — กลไกภายในของ HLS
 - บทที่ 4 — ภาษาต้นทางและ Coding Style
 - บทที่ 5 — Directives / Pragmas ที่ใช้บ่อย
 - บทที่ 6 — Interface Synthesis

Introduction to High-Level Synthesis

หลักสูตรอบรมระดับเริ่มต้นถึงระดับกลาง (Beginner → Intermediate)

กลุ่มเป้าหมาย: นักศึกษาวิศวกรรมชั้นปีที่ 3–4 และวิศวกรจบใหม่ที่มีพื้นฐาน RTL (Verilog/SystemVerilog) และเคยใช้งาน Open-source EDA (Yosys / OpenROAD / Icarus / COCOTB)

ระยะเวลา: 3 วัน (ทฤษฎี 1 วัน + ปฏิบัติ 2 วัน) — ปรับได้ตามความเหมาะสม

เครื่องมือหลัก: Bambu HLS (open-source), Yosys, OpenROAD, Icarus Verilog, COCOTB

PDK อ้างอิง: IHP SG13G2 (130 nm SiGe BiCMOS) และ SKY130

สารบัญ

Part I — ภาคทฤษฎี

- บทที่ 1 — High-Level Synthesis คืออะไร?
- บทที่ 2 — กระบวนการทำงานของ HLS
- บทที่ 3 — กลไกภายในของ HLS: Scheduling, Binding, Allocation
- บทที่ 4 — ภาษาต้นทางและ Coding Style
- บทที่ 5 — Directives / Pragmas ที่ใช้บ่อย
- บทที่ 6 — Interface Synthesis

Part II — ภาคปฏิบัติ (อยู่ในไฟล์ `hls-course-part2-practice.md`)

- การติดตั้ง Bambu HLS
- Lab 1 — Simple Adder
- Lab 2 — FIR Filter with Pipelining

10. Lab 3 — Matrix Multiplication

11. Lab 4 — Dataflow Streaming

12. การตรวจสอบ (Verification)

13. การนำไปใช้กับ OpenROAD (IHP SG13G2 / SKY130)

Part I — ภาคทฤษฎี

บทที่ 1 — High-Level Synthesis คืออะไร?

1.1 นิยาม

High-Level Synthesis (HLS) คือ กระบวนการแปลงภาษาระดับสูง (C / C++ / SystemC / MATLAB) ไปเป็น Register-Transfer Level (RTL) ที่อธิบายด้วย HDL (Verilog / VHDL) โดยอัตโนมัติ ซึ่งต่างจากการเขียน RTL ด้วยมือ (manual RTL design) ที่นักออกแบบต้องระบุทุก clock cycle, ทุก state, ทุก register ด้วยตนเอง

สรุปสั้น ๆ: HLS = “อธิบายว่าทำอะไร (What) → ให้เครื่องมือคิดว่าทำอย่างไร (How)”

1.2 เปรียบเทียบ: RTL แบบดั้งเดิม vs HLS

มิติ	RTL Design (Manual)	High-Level Synthesis
ภาษา	Verilog / VHDL / SystemVerilog	C / C++ / SystemC
ระดับนามธรรม	Cycle-accurate	Algorithmic / behavioral
ความเร็วในการออกแบบ	ช้า (วัน-สัปดาห์ต่อบล็อก)	เร็ว (ชั่วโมง)
ควบคุม micro-architecture	เต็มที่	ผ่าน directives/pragmas
ความยากของ design space exploration	ต้องเขียน RTL หลายเวอร์ชัน	เปลี่ยน directive แล้วรันใหม่
ผลลัพธ์เชิงคุณภาพ (QoR)	ขึ้นกับความเชี่ยวชาญ	ดีใกล้เคียงกันเมื่อปรับ directive เหมาะสม
การ verify	ต้องเขียน testbench HDL	ใช้ C testbench เดิมได้เลย (C/RTL co-sim)

1.3 ประวัติศาสตร์ (ย่อ)

- 1980s — งานวิจัย HLS เริ่มต้น (Synopsys Behavioral Compiler, Intel/Synopsys/Academic prototypes)
- 1990s — การใช้งานจริงในอุตสาหกรรม DSP
- 2000s — SystemC กลายเป็นมาตรฐาน IEEE 1666
- 2010s — Xilinx Vivado HLS / Vitis HLS ทำให้ HLS เป็น mainstream สำหรับ FPGA
- 2020s — Open-source HLS (Bambu, Dynamic) + การผสานกับ OpenROAD ทำให้ ASIC design เข้าถึงได้

1.4 เครื่องมือ HLS ที่ใช้ในอุตสาหกรรมและงานวิจัย

เครื่องมือ	ผู้พัฒนา	License	ใช้กับ
Vitis HLS	AMD/Xilinx	Commercial (free Webpack)	FPGA
Intel HLS Compiler	Intel	Commercial (free edition)	FPGA
Catapult HLS	Siemens EDA (Mentor)	Commercial	ASIC / FPGA
Stratus HLS	Cadence	Commercial	ASIC / FPGA
Bambu	Politecnico di Milano	Open-source (GPL)	ASIC / FPGA / OpenROAD
Dynamic	EPFL	Open-source (Apache 2.0)	ASIC (research)
LegUp	Univ. of Toronto	Open-source (legacy)	FPGA (research)
HLS4ML	CERN/CMU	Open-source (BSD)	ML on FPGA

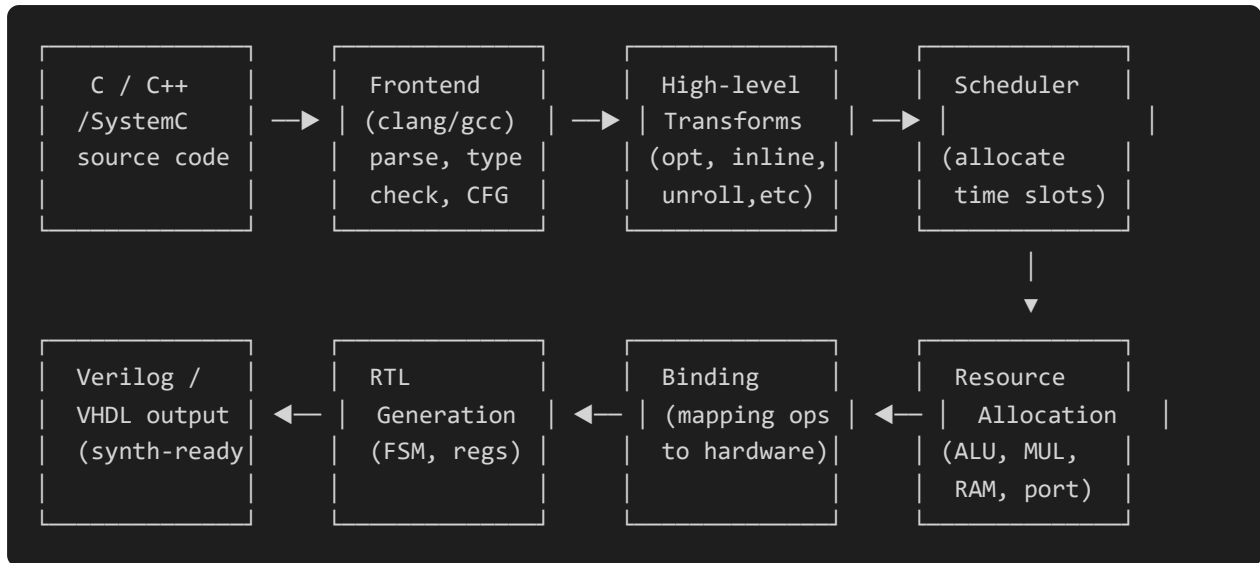
สำหรับหลักสูตรนี้ เราจะใช้ **Bambu HLS** เนื่องจากเป็น open-source และเชื่อมต่อกับ Yosys + OpenROAD ได้โดยตรง (เหมาะกับ PDK IHP SG13G2 / SKY130) แต่จะอธิบายแนวคิดในเชิง tool-agnostic เพื่อให้นำไปใช้กับเครื่องมือ commercial ได้ด้วย

1.5 ทำไมต้องเรียน HLS?

1. **Productivity** — เขียนน้อยลง 10x เมื่อเทียบกับ manual RTL
2. **Design Space Exploration (DSE)** — ลองหลาย micro-architecture ได้เร็ว
3. **Reuse** — testbench เดียวกัน verify ทั้ง C model และ RTL
4. **Algorithm Focus** — วิศวกรมุ่งเน้น “ทำอะไร” ไม่ใช่ “ทำอย่างไร”
5. **Open-source Ecosystem** — สอดคล้องกับ flow Yosys + OpenROAD ที่ใช้อยู่แล้ว

บทที่ 2 — กระบวนการทำงานของ HLS

2.1 Flow ภาพรวม



2.2 Frontend (ส่วนหน้า)

ใช้ compiler infrastructure (clang, gcc, **혹은** SystemC simulator) เพื่อ:

- Lexical/Syntax Analysis — แยก token, parse เป็น AST
- Type Checking — ตรวจสอบชนิดข้อมูล
- Control Flow Graph (CFG) — แทน execution path ของโปรแกรม
- Data Flow Analysis — ติดตามการใช้ตัวแปร (def-use chain)

ผลลัพธ์: Intermediate Representation (IR) เช่น LLVM IR หรือ IR เฉพาะของเครื่องมือ

2.3 High-Level Transformations

ก่อน scheduling เครื่องมืออาจทำ optimization:

- Function Inlining — แทน call ด้วย body เพื่อให้ scheduler เห็นภาพรวม
- Loop Transformations — unroll, fusion, fission, interchange, tiling
- Dead Code Elimination — ลบโค้ดที่ไม่ได้ใช้
- Constant Propagation — แทนค่าคงที่ตอนคอมไพล์

2.4 Scheduling — จัดสรร “เวลา”

Scheduling ตัดสินใจว่าแต่ละ operation จะทำงานใน clock cycle ใด โดย:

- เคารพ data dependencies (ไม่ให้ใช้ค่าก่อนผลิต)

- เคารพ **resource constraints** (ไม่เกินจำนวน functional unit ที่มี)
- พยายามบรรลุ **target latency / throughput**

อัลกอริทึมที่นิยม: - **ASAP** (As Soon As Possible) - **ALAP** (As Late As Possible) - **List Scheduling** (priority-based, เร็วและนิยมที่สุด) - **Force-Directed Scheduling**

2.5 Allocation & Binding — จัดสรร “ทรัพยากร”

- **Allocation** — เลือกว่าจะมี functional unit อะไรบ้าง เช่น 2x adder, 1x multiplier
- **Binding** — map operation แต่ละตัวไปยัง functional unit ตัวใด
- **Register Allocation** — เลือกว่าตัวแปรใดอยู่ใน register ตัวใด รวมถึง register sharing

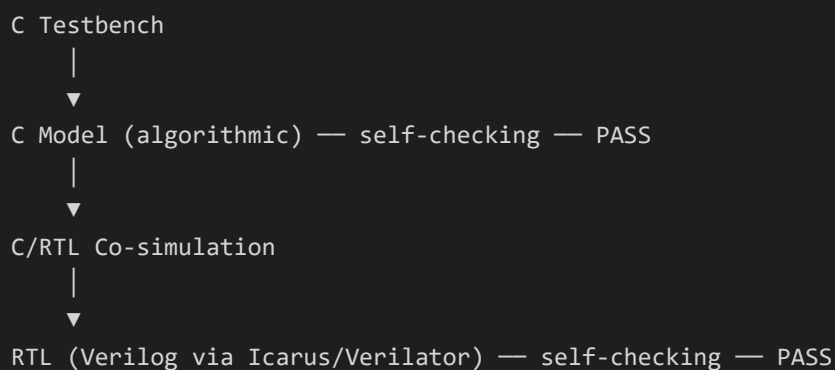
2.6 RTL Generation

สร้างไฟล์ Verilog/VHDL:

- **FSM** จาก CFG (state register, next-state logic)
- **Datapath** (functional units, muxes, wires)
- **Control Unit** (FSM + interface protocol)
- **Memory** (registers, BRAM inference, FIFO)
- **Interface** (ap_start, ap_done, AXI, FIFO, RAM ports)

2.7 Verification Flow (สำคัญมาก)

HLS มี co-simulation เป็นจุดแข็ง:

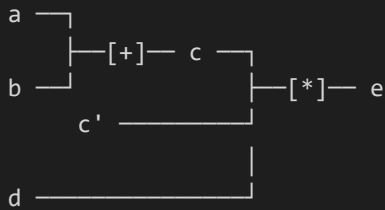


ข้อดี: ใช้ testbench ตัวเดียว ไม่ต้องเขียน HDL testbench ใหม่

บทที่ 3 — กลไกภายในของ HLS

3.1 Control-Data Flow Graph (CDFG)

HLS แทนโปรแกรมด้วย CDFG: - **Control Flow** — basic blocks + branches - **Data Flow** — operations + data dependencies



3.2 ตัวอย่าง: Scheduling แบบง่าย

โค้ด:

```
c = a + b;
e = c * d;
f = e + 1;
```

สมมติ adder ใช้ 1 cycle, multiplier ใช้ 2 cycles:

Cycle	0	1	2	3
Op1 +	✓			
Op2 *		✓	✓	
Op3 +				✓

ผลลัพธ์: Latency = 4 cycles, 1 adder, 1 multiplier

ถ้าเราเพิ่ม multiplier อีก 1 ตัว (resource constraint เปลี่ยน) → schedule จะสั้นลงได้

3.3 Resource Allocation & Binding

ตัวอย่างการ binding:

```
Op1: a + b    → adder_0
Op3: e + 1    → adder_0    (register share กับ Op1)
Op2: c * d    → mult_0
```

Cost tradeoffs: - Sharing functional unit → ใช้พื้นที่น้อย แต่ latency เพิ่ม - ไม่ share → เร็วขึ้น แต่พื้นที่เพิ่ม

3.4 Register Allocation

ตัวแปรที่ “มีชีวิต” (live) พร้อมกันต้องอยู่คนละ register:

```
int a, b, c, d, e;
c = a + b;    // a,b,c live
e = c + d;    // a,b dead; c,d,e live
```

Solution: graph coloring algorithm (เช่น Chaitin’s algorithm)

3.5 Memory Architecture

HLS ต้องตัดสินใจว่า array จะเก็บที่ไหน: - **Register** — เร็วสุด แต่ใช้ FF จำนวนมาก - **Distributed RAM (LUTRAM)** — เร็ว, จำกัดขนาด - **Block RAM (BRAM)** — ใหญ่, 1–2 cycle latency - **External memory** — ช้า, ใช้ bus protocol

ตัวเลือกนี้ถูกควบคุมด้วย **directive** (ดูบทที่ 5)

บทที่ 4 — ภาษาต้นทางและ Coding Style

4.1 ภาษาที่รองรับ

ภาษา	Bambu	Vitis HLS	Catapult
C	✓	✓	✓
C++	✓	✓	✓
SystemC	✓ (จำกัด)	ผ่าน wrapper	✓
OpenCL	—	✓	✓
MATLAB/Simulink	—	ผ่าน HDL Coder	✓

4.2 Arbitrary Precision Data Types

ภาษา C ใช้ int (32-bit), double (64-bit) ซึ่งไม่ตรงกับ hardware จริงเสมอ HLS จึงมี type พิเศษ:

Xilinx (Vitis HLS):

```
#include "ap_int.h"
ap_int<8> x;    // signed 8-bit
ap_uint<16> y;  // unsigned 16-bit
ap_fixed<16,8> fx; // Q8.8 fixed-point
```


Bambu (ac_types — Mentor/Siemens):

```
#include "ac_int.h"
ac_int<8,false> x;    // unsigned 8-bit
ac_int<8,true> y;     // signed 8-bit
ac_fixed<16,8,true,AC_RND,AC_SAT> fx; // fixed-point
```

เคล็ดลับ: ใช้ type เล็กที่สุดเท่าที่จำเป็น → ลด area, เร็วขึ้น

4.3 hls::stream (FIFO Channel)

สำหรับ streaming interface:

```
#include "hls_stream.h"
void producer(hls::stream<int>& out) {
    out.write(42);
}
void consumer(hls::stream<int>& in) {
    int x = in.read();
}
```

4.4 Coding Style สำหรับ HLS

หลักการสำคัญ:

- หลีกเลี่ยง dynamic memory (malloc, free, new) — HLS ต้องการ static memory
- หลีกเลี่ยง recursion — HLS ไม่ synthesize ได้ (Bambu/Vitis จะ error)
- หลีกเลี่ยง system calls (printf, scanf) — ใช้เฉพาะใน testbench
- ใช้ const — ช่วยให้ constant propagation ทำงาน
- Loop bounds ต้องรู้ตอน compile-time (อย่างน้อยสำหรับ unroll)
- หลีกเลี่ยง pointer arithmetic ที่ซับซ้อน — HLS วิเคราะห์ไม่ได้

ตัวอย่าง: Bad vs Good

```
// ❌ Bad: dynamic bound
for (int i = 0; i < n; i++) { ... } // n มาจาก input → ต้องระบุ max

// ✅ Good: known bound (หรือใช้ #pragma)
for (int i = 0; i < 16; i++) { ... }
```

```
// ❌ Bad: dynamic memory
int *buf = malloc(N * sizeof(int));

// ✅ Good: static array
int buf[1024];
```

```
// ❌ Bad: pointer alias
void foo(int *a, int *b) { *a = *b + 1; }
// HLS อาจ assume a และ b ซ้ำที่เดียวกัน (alias) → schedule ไม่กล้า parallel

// ✅ Good: ใช้ restrict
void foo(int * restrict a, int * restrict b) { *a = *b + 1; }
```

4.5 Function Hierarchy & Top Function

HLS เลือก top function เป็นจุดเริ่มต้น synthesis:

```
// top function – argument จะกลายเป็น port
void dut(int a, int b, int& c) {
    #pragma HLS PIPELINE
    c = a + b;
}

int main() { // ไม่ synthesize
    int a=1, b=2, c;
    dut(a, b, c);
    return (c == 3) ? 0 : 1;
}
```

Vitis HLS ใช้ `#pragma HLS INTERFACE` ระบุ port protocol Bambu ใช้ `--top-function` flag บน command line

บทที่ 5 — Directives / Pragmas ที่ใช้บ่อย

คำสั่งเหล่านี้บอก HLS ว่า “อยากให้ micro-architecture เป็นอย่างไร” ใน Vitis HLS ใช้ `#pragma HLS ...` ใน Bambu ใช้ `#pragma HLS ...` เช่นกัน หรือระบุผ่าน flag `--pragma=...`

5.1 PIPELINE

วัตถุประสงค์: เร่ง throughput — ทำให้ loop ใหม่เริ่มได้ทุก ๆ N cycles (II = Initiation Interval)

```
void foo(int a[16], int b[16], int c[16]) {
#pragma HLS PIPELINE II=1
    for (int i = 0; i < 16; i++) {
        c[i] = a[i] + b[i];
    }
}
```

ผลลัพธ์: - ก่อน PIPELINE: 16 iterations × (latency) = ~16+ cycles - หลัง PIPELINE II=1: 1 iteration / clock → 16 cycles + pipeline fill/drain

ค่า II: - II=1 คือเป้าหมายสูงสุด (เร็วสุด) - ถ้า II ต่ำกว่าที่เป็นไปได้ HLS จะบอก warning

5.2 UNROLL

วัตถุประสงค์: สร้าง hardware ซ้ำหลายชุดเพื่อทำงานขนาน

```
#pragma HLS UNROLL factor=4
for (int i = 0; i < 16; i++) {
    c[i] = a[i] * b[i];
}
```

- **UNROLL** (ไม่มี factor) = unroll เต็ม
- **UNROLL factor=4** = unroll 4 ครั้ง (4x parallel units)
- Cost: พื้นที่เพิ่ม, Benefit: throughput เพิ่ม

5.3 ARRAY_PARTITION

วัตถุประสงค์: แบ่ง array เพื่อให้เข้าถึงหลาย element พร้อมกันได้

```
int buf[16];
#pragma HLS ARRAY_PARTITION variable=buf complete
// ตอนนี้ buf[0]..buf[15] อยู่คนละ register/RAM → เข้าถึงพร้อมกันได้
```

Modes: - **complete** — แยกทุก element - **block factor=N** — แบ่งเป็น block ละ N - **cyclic factor=N** — สลับแบบ round-robin

5.4 DATAFLOW

วัตถุประสงค์: ทำให้ function หลาย ๆ ตัวทำงานพร้อมกัน (task-level parallelism)

```
void kernel(in_t a, in_t b, out_t& c) {
#pragma HLS DATAFLOW
    stage1(a, b, t1);
    stage2(t1, t2);
    stage3(t2, c);
}
```

ผลลัพธ์: pipeline ระหว่าง function ใช้ hls::stream เชื่อมต่อ

5.5 INLINE

วัตถุประสงค์: เอา body ของ function มาแทนที่ call site

```
#pragma HLS INLINE
int square(int x) { return x*x; }
```

เมื่อไหร่ใช้: เมื่อ function เล็กและต้องการ share resource กับ caller

5.6 LOOP_FLATTEN / LOOP_MERGE

```
#pragma HLS LOOP_FLATTEN
for (int i = 0; i < 4; i++)
    for (int j = 0; j < 4; j++)
        ... ;
// → กลายเป็น Loop เดียว 16 iterations
```

Benefit: เพิ่ม optimization opportunity

5.7 DEPENDENCE

บอก HLS ว่าไม่มี dependency (เมื่อ HLS อนุมานว่ามี):

```
#pragma HLS DEPENDENCE variable=a inter false
// ไม่มี dependency ระหว่าง iteration ของ array a
```

5.8 สรุป Directives

Directive	ผลต่อ Area	ผลต่อ Latency	ใช้บ่อยที่ไหน
PIPELINE	+	-	inner loop
UNROLL	++	-	loop เล็ก
ARRAY_PARTITION	++	-	memory bottleneck
DATAFLOW	+	-	pipeline function
INLINE	-	+	small helper
LOOP_FLATTEN	+/-	-	nested loop
DEPENDENCE	0	-	false dep

บทที่ 6 — Interface Synthesis

6.1 Block-Level Interface (Default)

HLS สร้าง port protocol เริ่มต้น:

Port	ความหมาย
ap_start	เริ่มทำงาน (input, pulse)
ap_done	ทำงานเสร็จ (output, pulse)
ap_idle	ยังไม่เริ่ม/ทำเสร็จแล้ว (output, level)
ap_ready	พร้อมรับ input ใหม่ (output, level)

6.2 Function Argument → Port

```
void dut(int a, int b, int& c);  
// → ports: a (input), b (input), c (output)
```

โดย default ใช้ handshake protocol: - a_in (data), a_vld (valid), a_rd (ready) — หรือ variant อื่น

6.3 AXI Interface

Vitis HLS / Bambu สามารถสร้าง AXI4, AXI4-Lite, AXI4-Stream:

```
#pragma HLS INTERFACE m_axi port=a bundle=A depth=1024
#pragma HLS INTERFACE s_axilite port=return bundle=CTRL
```

AXI4-Stream (สำหรับ streaming data):

```
#pragma HLS INTERFACE axis port=in_stream
#pragma HLS INTERFACE axis port=out_stream
```

6.4 FIFO / Stream Interface

```
#include "hls_stream.h"
void dut(hls::stream<int>& in, hls::stream<int>& out) {
#pragma HLS INTERFACE axis port=in
#pragma HLS INTERFACE axis port=out
    out.write(in.read() + 1);
}
```

6.5 RAM Interface

```
#pragma HLS INTERFACE bram port=ram
```

HLS จะ infer BRAM (block RAM) โดยอัตโนมัติ — ใช้สำหรับ array ที่ใหญ่

6.6 Memory Mapped vs Streaming

	Memory-Mapped (AXI)	Streaming (FIFO)
Address	มี	ไม่มี
Latency	สูง (bus)	ต่ำ
Throughput	จำกัดด้วย bus	ต่อ clock
ใช้เมื่อ	CPU เข้าถึง	data pipeline

ดู Part II สำหรับการปฏิบัติกับ Bambu HLS, การ verify, และ integration กับ OpenROAD